

Iphe Hash Algorithm

Submitter: Iphe Algorithm Group

June 30, 2026

Contents

1	Introduction	3
2	Algorithm Description	3
2.1	Parameterized Definition	3
2.2	Submitted Instances	4
2.3	Padding Rule	7
3	Main Sponge-F Process	7
3.1	Initialization and Message Blocking	9
3.2	Absorbing Phase	9
3.3	Squeezing Phase	9
4	Permutation Function	10
4.1	AC: Add Constants	10
4.1.1	Round-Constant Generation	10
4.2	GM: Group Mixing	12
4.3	WR: Cyclic Rotation	14
4.4	WP: Word Permutation	14
5	Design Rationale	14
5.1	The Overall Structure	14
5.2	Security Against Modular Differential Attack	16
5.3	Security of Sponge-F	16
5.4	Variable-Rate Parameters	16
5.5	Multi-Scale Small-Round	16
5.6	Tree-Structured Group Mixing and Invertibility	17
5.7	Constants in Permutation Function	18
5.8	Innovation of GM Operation	18
5.9	Simplicity of ARX structure	19
5.10	Flexibility of Parameters	19
5.11	Implementation Features	19

6	Security Analysis	20
6.1	Security Analysis of the Overall Structure	20
6.2	Security Analysis of the Permutation Function	21
6.2.1	Differential, Linear, and Differential-Linear Statistical Analysis . .	21
6.2.2	Differential-Neural Analysis	23
6.2.3	Distinguishing Attacks Based on Algebraic Degree	23
6.3	Preimage and Second-Preimage Security Analysis	25
6.4	Collision Security Analysis	27
6.5	Statistical Randomness Tests	28
7	Implementation	29
7.1	Overview	29
7.2	Software Implementation	30
7.3	Theoretical Cost	30
7.4	Benchmark Environment	31
7.5	Software Benchmark Results	33
7.6	Resource Consumption	33

1 Introduction

Hash functions are important cryptographic primitives widely used in applications such as digital signatures and pseudorandom number generators. In 2025, the Institute of Commercial Cryptography Standards (ICCS) launched the Next-generation Commercial Cryptographic Algorithms Program (NGCC) to solicit new hash function algorithms with improved security, high performance, and novel features.

The sponge structure [1, 2] was introduced by Guido et al. It uses a permutation function as the core component of the hash function and divides the state words into a rate part and a capacity part. In this design, message words are XORed into the rate part, and output hash values are also derived from it, while the capacity part remains hidden to ensure security. Many well-known hash functions are built on this structure, including Keccak [3], Ascon-Hash [4], and Gimli-Hash [5].

Feed-forward Sponge-F structure [6] was designed by Guo et al. In this structure, after padding, the input message is split into r -bit blocks and absorbed by XORing each block into the rate part of the state. Unlike the ordinary Sponge construction [1, 2], Sponge-F saves the capacity part before each permutation call and XORs it back into the state after the permutation. The squeezing phase also outputs digest bits from the capacity part. Thus, the capacity part not only serves as hidden state that is not directly overwritten by message-block absorption, but also contributes to state updates across permutation calls through feed-forward.

The adoption of this outer structure is supported by existing security analyses. Guo et al. [6] present the Sponge-F structure and establish collision and (second-)preimage security bounds in both classical and quantum settings in the random-permutation model. Sun et al. [7] independently propose similar feed-forward sponge constructions, SPONGE-DM and SPONGE-EDM, and prove their improved preimage-security properties. The outer layer used in this document adopts the Sponge-F structure, and its security bounds form the basis for the structural security claim in Section 6.1.

In this paper, we present Iphe, a new cryptographic hash function. Iphe has output bitlength 512, 768 and 1024, with a simple Sponge-F structure. We devise a novel permutation function that only depend on arithmetic operations on 64 bit words, without using any S-box. Our permutation function has a tree-based mixing structure, and multi-scale word permutation. The evaluation shows that our functions are fast in software, and secure under our analysis model. Our design is a practical and fast cryptographic hash function.

2 Algorithm Description

2.1 Parameterized Definition

A member of the Iphe family is written as

$$Iphe[w, n, d, \tau, N_r],$$

where w is the word size, n is the number of words in the state, d is the digest length, τ is the capacity overhead, and N_r is the number of large rounds. The capacity overhead is chosen to cover the maximum supported number of message blocks L , namely

$$\tau \geq \lceil \log_2 L \rceil.$$

The remaining parameters are derived from these values:

$$b = nw, \quad n_r = \log_2 n, \quad c = d + \tau, \quad r = b - c.$$

Thus n is assumed to be a power of two, and the parameters must satisfy

$$w \geq n, \quad 2 \mid w, \quad w \mid d, \quad w \mid \tau, \quad c < b.$$

These conditions ensure that both the rate and capacity parts consist of whole w -bit words.

The state is denoted by

$$S = S_0 \parallel S_1 \parallel \cdots \parallel S_{n-1},$$

where each S_i is a w -bit word. The first r/w words form the rate part, and the remaining c/w words form the capacity part. The capacity mask is

$$M_c = 0^r \parallel 1^c.$$

After padding (see Section 2.3), the padded message m' has bit length divisible by r . Let $t = |m'|/r$. Split it as

$$m' = B^{(0)} \parallel B^{(1)} \parallel \cdots \parallel B^{(t-1)}, \quad |B^{(i)}| = r.$$

Each message block is further split into r/w words:

$$B^{(i)} = B_0^{(i)} \parallel B_1^{(i)} \parallel \cdots \parallel B_{r/w-1}^{(i)}.$$

2.2 Submitted Instances

The submitted Iphe instances use

$$w = 64, \quad n = 32, \quad \tau = 64, \quad N_r = 5,$$

and support $d \in \{512, 768, 1024\}$. Therefore,

$$b = 2048, \quad n_r = \log_2 n = 5, \quad c = d + \tau = d + 64, \quad r = 1984 - d.$$

Iphe supports messages of length at most $2^{64} - 1$ bits. Under the submitted instances, the maximum supported number of message blocks satisfies $\log_2 L < 55$, so the choice $\tau = 64$ covers the required overhead. Equivalently,

$$\begin{aligned} \text{Iphe-512} &= \text{Iphe}[64, 32, 512, 64, 5], \\ \text{Iphe-768} &= \text{Iphe}[64, 32, 768, 64, 5], \\ \text{Iphe-1024} &= \text{Iphe}[64, 32, 1024, 64, 5]. \end{aligned}$$

The rotation constants are defined by

$$\rho_i = \left(((7i) \bmod n) + \frac{w-n}{2} \right) \bmod w, \quad 0 \leq i < n.$$

For the submitted instances, this gives $\rho_i = ((7i) \bmod 32) + 16$.

Table 1: Parameters of the submitted Iphe instances

Symbol	Value	Meaning
w	64	Number of bits in each word
n	32	Number of words in the state
b	$nw = 2048$	Total number of state bits
d	512, 768, 1024	Output digest length
τ	64	Capacity overhead
c	$d + \tau = d + w$	Capacity size
r	$b - c = 1984 - d$	Rate size
N_r	5	Number of large rounds in the permutation
n_r	$\log_2 n = 5$	Number of small rounds per large round
ρ_i	$((7i) \bmod 32) + 16$	Left-rotation constant for word i

Table 2: Relationship between d , c , and r

Output length d	Capacity bits c	Rate bits r
512	576	1472
768	832	1216
1024	1088	960

Endian We take bit strings as inputs and write ε for the empty string. To avoid endianness ambiguity across implementations, this document interprets bit strings, bytes, and 64-bit words as follows.

- A bit string is split into bytes from left to right, in the way that consecutive bits $x_0, x_1, \dots, x_7$ are converted to a byte, according to the least significant bit first rule. Hence the byte value is

$$x_0 2^0 + x_1 2^1 + \dots + x_7 2^7.$$

- Every 8 consecutive bytes form one 64-bit word in little-endian order. If the byte values are $a_0, a_1, \dots, a_7$, the corresponding 64-bit word value is

$$a_0 + a_1 2^8 + \dots + a_7 2^{56}.$$

Equivalently, consecutive bits $x_0, x_1, \dots, x_{63}$ correspond to the word value

$$\sum_{j=0}^7 \sum_{k=0}^7 x_{8j+k} 2^{8j+k}.$$

This convention is consistent with the representation of unsigned 64-bit integers on common little-endian platforms.

- When output is converted to a byte string, the inverse rule is used: a 64-bit word X is output as $a_0, a_1, \dots, a_7$, where

$$a_j = \lfloor X/2^{8j} \rfloor \bmod 2^8, \quad 0 \leq j < 8.$$

Each byte is then mapped to output bits from least significant bit to most significant bit. Hexadecimal strings in test vectors are written from left to right in output-byte order.

Operators We use the following operators:

- $x \boxplus y$: addition of two w -bit words modulo 2^w .
- $x \oplus y$: bitwise XOR.
- $x \wedge y$: bitwise AND.
- $\langle x, y \rangle$: inner product of two bit strings. For $|x| = |y| = l$, it is defined as $\langle x, y \rangle := \bigoplus_{i=0}^{l-1} x[i] \wedge y[i]$.
- $x \ll \rho$: ordinary left shift of the w -bit word x by ρ bits; high bits are discarded and low bits are filled with zeros.
- $x \lll \rho$: cyclic left rotation of the w -bit word x by ρ bits.
- $\parallel$: concatenation of bit strings or word strings.

Input and Output Interfaces For each parameter set $Iphe[w, n, d, \tau, N_r]$, the input is a message m of length at most $2^{64} - 1$ bits, and the output is a d -bit digest. We process the input and output as byte arrays for KAT generation. For a message bit length not divisible by 8, the valid bits of the last byte occupy the high bits. Internally, Iphe interprets bits in least-significant-bit-first order within each byte, so the implementation reverses the bit order in each input byte. No additional bit reversal is applied to output bytes.

2.3 Padding Rule

Let the message m have length ℓ bits. For the selected parameter set, the padding rule uses the rate r . The padding function is defined as follows:

1. Append one bit 1 after m .
2. Append k bits 0, where k is the smallest nonnegative integer satisfying

$$\ell + 1 + k + 1 \equiv 0 \pmod{r}.$$

3. Append one more bit 1.

The padded message is denoted by m' . By construction, $|m'|$ is an integer multiple of r .

3 Main Sponge-F Process

For a parameter set $Iphe[w, n, d, \tau, N_r]$, the hash function $\text{Hash}(m; w, n, d, \tau, N_r)$ uses the Sponge-F construction [6] as the outer iteration structure and a permutation function f , which operates on a b -bit internal state. The derived parameters b , n_r , c , and r are defined in the previous section.

Figure 1 gives a schematic view of the Sponge-F structure used in this document, highlighting the two main differences from the ordinary sponge construction: the feed-forward path for the capacity part (C_i) and output from the capacity part during squeezing [6].

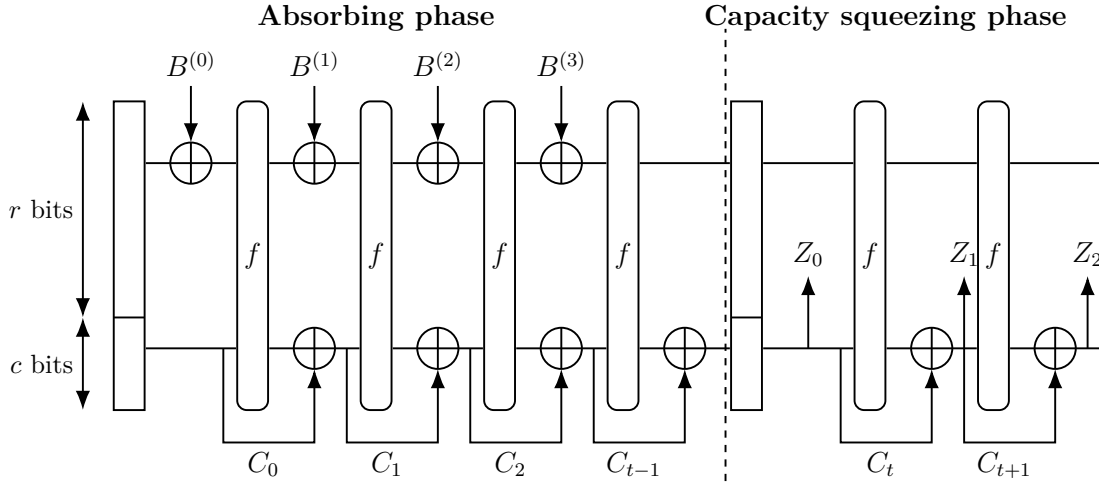


Figure 1: Sponge-F saves the capacity part C_i before each permutation call, and XORs it back into the capacity part after the permutation. It also squeezes output blocks Z_i from the capacity part.

Algorithm 1: Main Sponge-F hash process for $Iphe[w, n, d, \tau, N_r]$

Input: Message bit string m , parameter set $Iphe[w, n, d, \tau, N_r]$, permutation function f

Output: d -bit digest

// Initialization

$S \leftarrow 0^b;$

$H \leftarrow \varepsilon;$

// Padding and block splitting

$m' \leftarrow \text{Padding}(m);$

$t \leftarrow |m'|/r;$

Split m' into $m' = B^{(0)} \parallel B^{(1)} \parallel \dots \parallel B^{(t-1)}$, where $|B^{(i)}| = r;$

Split each $B^{(i)}$ into $B_0^{(i)} \parallel B_1^{(i)} \parallel \dots \parallel B_{r/w-1}^{(i)}$, where each $B_j^{(i)}$ is a w -bit word;

// Absorbing phase

for $i \leftarrow 0$ **to** $t - 1$ **do**

for $j \leftarrow 0$ **to** $r/w - 1$ **do**

$S_j \leftarrow S_j \oplus B_j^{(i)};$

end

$C \leftarrow S \wedge M_c;$

$S \leftarrow f(S);$

$S \leftarrow S \oplus C;$

end

// Squeezing phase

while $|H| < d$ **do**

$H \leftarrow H \parallel S_{r/w} \parallel S_{r/w+1} \parallel \dots \parallel S_{n-1};$

if $|H| \geq d$ **then**

return $H[0 : d - 1];$

end

$C \leftarrow S \wedge M_c;$

$S \leftarrow f(S);$

$S \leftarrow S \oplus C;$

end

return $H;$

3.1 Initialization and Message Blocking

For the selected parameter set, initialize

$$S \leftarrow 0^b.$$

Then compute $m' = \text{Padding}(m)$, where the padding rule uses the rate r , and split m' into r -bit blocks:

$$m' = B^{(0)} \parallel B^{(1)} \parallel \dots \parallel B^{(t-1)}.$$

3.2 Absorbing Phase

For $i = 0, 1, \dots, t - 1$, perform the following steps:

1. For $j = 0, 1, \dots, r/w - 1$, set

$$S_j \leftarrow S_j \oplus B_j^{(i)}.$$

This step XORs the i -th message block into the rate part of the state.

2. Record the current capacity part of the state:

$$C \leftarrow S \wedge M_c.$$

3. Apply the permutation:

$$S \leftarrow f(S).$$

4. Apply capacity feed-forward:

$$S \leftarrow S \oplus C.$$

3.3 Squeezing Phase

Initialize the output string $H \leftarrow \varepsilon$. While $|H| < d$, repeat:

1. Append the capacity part of the current state to the output string:

$$H \leftarrow H \parallel S_{r/w} \parallel S_{r/w+1} \parallel \dots \parallel S_{n-1}.$$

2. If $|H| \geq d$, output the first d bits of H and stop.

3. Otherwise, record the capacity part $C \leftarrow S \wedge M_c$, compute $S \leftarrow f(S)$, and set $S \leftarrow S \oplus C$.

Thus, both the absorbing and squeezing phases use the same feed-forward rule, and the digest is taken from the capacity part.

4 Permutation Function

For a parameter set $Iphe[w, n, d, \tau, N_r]$, the internal permutation is denoted by f_{w,n,N_r} . It depends on the word size w , the number of state words n , and the number of large rounds N_r ; the number of small rounds is the derived value $n_r = \log_2 n$.

The permutation is built from four gadgets, which form one small-round mixing function

$$\text{rnd} = \text{WP} \circ \text{WR} \circ \text{GM} \circ \text{AC}.$$

The full permutation applies the mixing function for N_r large rounds, with n_r small rounds in each large round:

$$f_{w,n,N_r} = \text{Rnd}^{N_r}, \quad \text{Rnd} = \text{rnd}^{n_r}.$$

For the submitted Iphe instances, $w = 64$, $n = 32$, $N_r = 5$, and $n_r = 5$, so the mixing function is executed 25 times. See Figure 2 for one small round. The full permutation is specified in Algorithm 2.

Round Index and Stride Large rounds are indexed by $I_r = 0, 1, \dots, N_r - 1$, and small rounds are indexed by $i_r = 0, 1, \dots, n_r - 1$. In small round i_r , the processing stride is $l = 2^{i_r+1}$. For the submitted parameter $n = 32$, the processing strides within one large round are 2, 4, 8, 16, 32 words. In general, every l consecutive state words form one group, and the state is split into n/l groups.

4.1 AC: Add Constants

In large round I_r and small round i_r , the state is split into n/l groups. For each group index $i = 0, 1, \dots, n/l - 1$, generate the constant arrays α , β , and γ as described in Section 4.1.1, and compute

$$\text{rc}[i, i_r, I_r] = \alpha[i] \oplus \beta[i_r] \oplus \gamma[I_r].$$

Then update the first word of the group:

$$S_{il} \leftarrow S_{il} \oplus \text{rc}[i, i_r, I_r].$$

All other words remain unchanged in the AC operation.

4.1.1 Round-Constant Generation

Round constants are generated from the mathematical constant π . Let

$$\pi_0, \pi_1, \dots, \pi_{n/2+n_r+N_r-1}$$

be a constant array, where

$$\pi_i = \left\lfloor \pi \cdot 2^{w(i+1)} \right\rfloor \bmod 2^w.$$

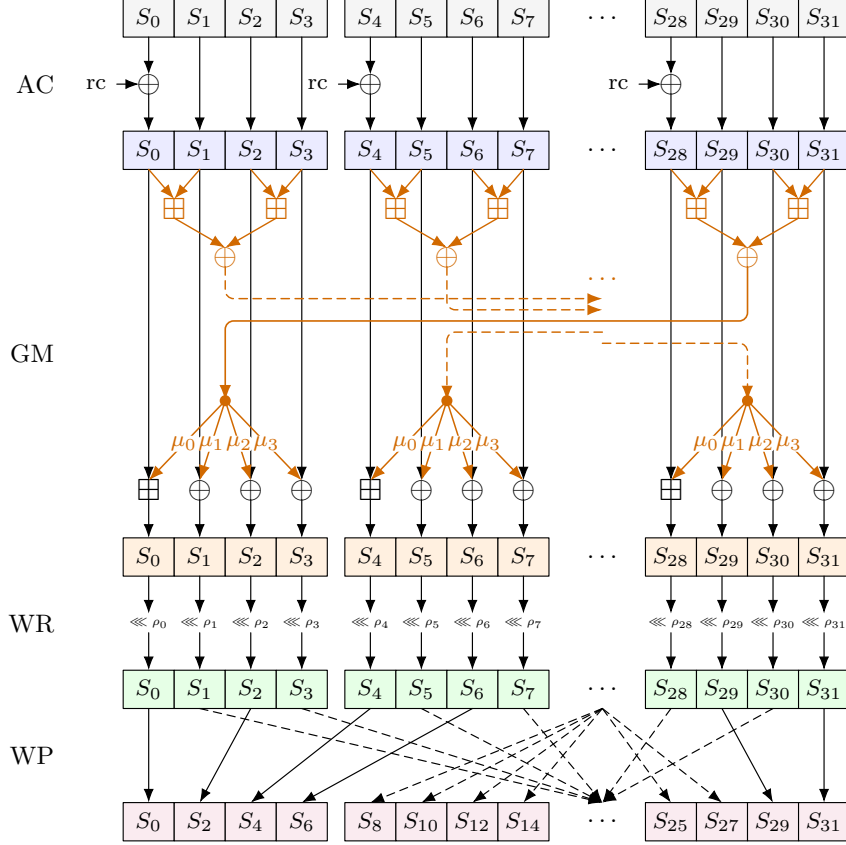


Figure 2: Operations in one small round of the permutation. The figure uses the recommended instance state size $n = 32$ and illustrates a small round with processing stride $l = 4$ (i.e., the small round where $i_r = 1$); the middle words $S_8, \dots, S_{27}$ are represented by ellipses. The solid orange arrow highlights one treeSum feedback, and $\oplus/\oplus$ show how the target group is updated; fixed-coefficient writeback uniformly uses $\mu_j = 1 + 2^{j+1}$ ($0 \leq j < l$).

We have the following constants,

$$\begin{aligned}\alpha[i] &\leftarrow \pi_i, & 0 \leq i < n/2, \\ \beta[i] &\leftarrow \pi_{n/2+i}, & 0 \leq i < n_r, \\ \gamma[i] &\leftarrow \pi_{n/2+n_r+i}, & 0 \leq i < N_r.\end{aligned}$$

Thus, for the submitted parameters, 16 α constants, 5 β constants, and 5 γ constants are required, for a total of 26 64-bit constant words. The concrete constant values are listed in Table 3.

Table 3: Round constants used in the AC operation

i	$\alpha[i]$	i	$\alpha[i]$
0	0x243f6a8885a308d3	8	0x9216d5d98979fb1b
1	0x13198a2e03707344	9	0xd1310ba698dfb5ac
2	0xa4093822299f31d0	10	0x2ffd72dbd01adfb7
3	0x082efa98ec4e6c89	11	0xb8e1afed6a267e96
4	0x452821e638d01377	12	0xba7c9045f12c7f99
5	0xbe5466cf34e90c6c	13	0x24a19947b3916cf7
6	0xc0ac29b7c97c50dd	14	0x0801f2e2858efc16
7	0x3f84d5b5b5470917	15	0x636920d871574e69
i	$\beta[i]$	i	$\gamma[i]$
0	0xa458fea3f4933d7e	0	0xc5d1b023286085f0
1	0x0d95748f728eb658	1	0xca417918b8db38ef
2	0x718bcd5882154aee	2	0x8e79dcb0603a180e
3	0x7b54a41dc25a59b5	3	0x6c9e0e8bb01e8a3e
4	0x9c30d5392af26013	4	0xd71577c1bd314b27

4.2 GM: Group Mixing

In large round I_r and small round i_r , let the processing stride be $l = 2^{i_r+1}$. The GM operation first computes a tree-structured mixing value for each source group. Let $\text{TS}_l(a)$ denote the treeSum starting at index a with length l . It is defined recursively by

$$\text{TS}_1(a) = S_a,$$

and for $l = 2^s$, $s \geq 1$,

$$\text{TS}_l(a) = \begin{cases} \text{TS}_{l/2}(a) \boxplus \text{TS}_{l/2}(a + l/2), & s \text{ is odd,} \\ \text{TS}_{l/2}(a) \oplus \text{TS}_{l/2}(a + l/2), & s \text{ is even.} \end{cases}$$

For the submitted $n = 32$ instances, this gives the following five nontrivial cases:

$$\begin{aligned} \text{TS}_2(a) &= \text{TS}_1(a) \boxplus \text{TS}_1(a+1), \\ \text{TS}_4(a) &= \text{TS}_2(a) \oplus \text{TS}_2(a+2), \\ \text{TS}_8(a) &= \text{TS}_4(a) \boxplus \text{TS}_4(a+4), \\ \text{TS}_{16}(a) &= \text{TS}_8(a) \oplus \text{TS}_8(a+8), \\ \text{TS}_{32}(a) &= \text{TS}_{16}(a) \boxplus \text{TS}_{16}(a+16). \end{aligned}$$

The target group is specified by a fixed mapping D_l . Let $q = \log_2(n/l)$, and let rev_q denote q -bit reversal. That is, if

$$i = \sum_{t=0}^{q-1} b_t 2^t, \quad b_t \in \{0, 1\},$$

then

$$\text{rev}_q(i) = \sum_{t=0}^{q-1} b_t 2^{q-1-t}.$$

For $q = 0$, define $D_l(0) = 0$. For $q > 0$, the target group index is

$$D_l(i) = \text{rev}_q((\text{rev}_q(i) + 1) \bmod 2^q).$$

Equivalently, D_l adds one to the q -bit group index with carry propagation from the most significant bit toward the least significant bit: starting at the most significant bit, consecutive 1 bits are set to 0, the first 0 bit encountered is set to 1, and the remaining lower bits are unchanged; the all-one group index maps to 0. This mapping is a permutation on the n/l group indices. For example, for the submitted $n = 32$ instance and $l = 4$, group indices $0, \dots, 7$ are mapped to $4, 5, 6, 7, 2, 3, 1, 0$.

Let

$$u_i = \text{TS}_l(il), \quad d = D_l(i)l.$$

Define the feedback coefficients by

$$\mu_j = 1 + 2^{j+1}, \quad 0 \leq j < l.$$

Here $\mu_j u_i$ denotes multiplication of the w -bit word u_i by the fixed constant μ_j modulo 2^w . GM updates the target group through a temporary array T :

$$T_d = S_d \boxplus (\mu_0 u_i),$$

$$T_{d+j} = S_{d+j} \oplus (\mu_j u_i), \quad 1 \leq j < l.$$

After all source groups have been processed, set $S_j \leftarrow T_j$. All S_j terms on the right-hand side refer to the state before entering GM. The fixed-coefficient multiplication for $j = 0$ can be implemented with two modular additions; for $j > 0$, it can be expanded as $u_i \boxplus (u_i \ll (j+1))$, where ordinary left shifts discard high bits.

4.3 WR: Cyclic Rotation

For each word S_i , where $i = 0, 1, \dots, n-1$, perform

$$S_i \leftarrow S_i \lll \rho_i, \quad \rho_i = \left(((7i) \bmod n) + \frac{w-n}{2} \right) \bmod w.$$

For the submitted instances, this reduces to $\rho_i = ((7i) \bmod 32) + 16$.

4.4 WP: Word Permutation

Let the temporary array be $T_0, T_1, \dots, T_{n-1}$. For $i = 0, 1, \dots, n-1$, if i is even, set

$$T_{i/2} \leftarrow S_i;$$

if i is odd, set

$$T_{n/2+(i-1)/2} \leftarrow S_i.$$

Finally set $S_i \leftarrow T_i$. Equivalently,

$$S'_i = \begin{cases} S_{2i}, & 0 \leq i < n/2, \\ S_{2(i-n/2)+1}, & n/2 \leq i < n. \end{cases}$$

5 Design Rationale

5.1 The Overall Structure

The construction consists of an existing Sponge-F outer structure [6] and an internal permutation f . The outer structure converts arbitrary-length messages into fixed-length digests, while the internal permutation provides nonlinearity, diffusion, and symmetry breaking among state words. If the internal permutation f is modeled as a random permutation, the outer structure provides the corresponding security proof, with the security strength depending on the capacity size, digest length, message length, and related parameters.

The components of the permutation f play distinct cryptographic roles. The AC operation XORs round constants to break inter-round and intra-word symmetries. The GM operation is the main component for confusion and diffusion: treeSum provides multi-word mixing inside a source group, modular-addition carries provide nonlinearity, cross-group feedback transfers the local mixing value to another target group, and fixed-coefficient writeback spreads the feedback bits within target words. The WR operation performs cyclic rotations within words, so position-dependent properties such as low-bit and high-bit behavior are moved to different bit ranges in later small rounds rather than remaining tied to fixed intra-word positions. The WP operation changes adjacency relations among words, so the group composition and same-group relations change in the next small round. After multiple rounds, these operations are intended to provide confusion and diffusion both within words and across words.

Algorithm 2: Permutation function $f_{w,n,N_r}(S)$

Input: b -bit state $S = S_0 \parallel S_1 \parallel \dots \parallel S_{n-1}$, round constants α, β, γ

Output: Permuted state S'

```
for  $I_r \leftarrow 0$  to  $N_r - 1$  do
  for  $i_r \leftarrow 0$  to  $n_r - 1$  do
     $l \leftarrow 2^{i_r+1}$ ;
    // AC: XOR round constants
    for  $i \leftarrow 0$  to  $n/l - 1$  do
       $rc \leftarrow \alpha[i] \oplus \beta[i_r] \oplus \gamma[I_r]$ ;
       $S_{il} \leftarrow S_{il} \oplus rc$ ;
    end
    Initialize temporary array  $Y_0, Y_1, \dots, Y_{n-1}$ ;
    for  $i \leftarrow 0$  to  $n/l - 1$  do
       $u \leftarrow TS_l(il)$ ;
       $d \leftarrow D_l(i) \cdot l$ ;
       $Y_d \leftarrow S_d \boxplus (\mu_0 u)$ ;
      for  $j \leftarrow 1$  to  $l - 1$  do
         $Y_{d+j} \leftarrow S_{d+j} \oplus (\mu_j u)$ ;
      end
    end
    for  $i \leftarrow 0$  to  $n - 1$  do
       $S_i \leftarrow Y_i$ ;
    end
    for  $i \leftarrow 0$  to  $n - 1$  do
       $S_i \leftarrow S_i \lll \rho_i$ ;
    end
    Initialize temporary array  $T_0, T_1, \dots, T_{n-1}$ ;
    for  $i \leftarrow 0$  to  $n - 1$  do
      if  $i \bmod 2 = 0$  then
         $T_{i/2} \leftarrow S_i$ ;
      else
         $T_{n/2+(i-1)/2} \leftarrow S_i$ ;
      end
    end
    for  $i \leftarrow 0$  to  $n - 1$  do
       $S_i \leftarrow T_i$ ;
    end
  end
end
return  $S_0 \parallel S_1 \parallel \dots \parallel S_{n-1}$ ;
```

5.2 Security Against Modular Differential Attack

Although we construct our internal permutation f in an ARX schedule, it has strong security against modular differential attack, which is usually an effective approach against ARX permutations. However, in differential attack, it is required to enforce conditions to satisfy the proper propagation of differences. Hence, for an attacker to ensure more conditions, the values of some states are to be fixed/controlled.

However, in the Sponge-F structure, message words cannot go through the rate part of the input state, and therefore the uncontrolled capacity part can affect the differential path very quickly. Hence, our structure are resistant to modular differential attacks.

5.3 Security of Sponge-F

We adopt the existing Sponge-F structure [6], which has existing security proofs in the random-permutation model. The security analysis gives parameter constraints among the capacity length, digest length, and maximum number of message blocks. Our design chooses the capacity parameters and states the outer-structure security claim according to those constraints; the design and analysis focus is the internal permutation f and its parameter instantiations.

According to the Sponge-F security proof [6], if the output digest length is d bits and the maximum supported number of message blocks is L , then the capacity part should have at least

$$c \geq d + \log_2 L$$

bits to achieve the intended security strength against collision, preimage, and second-preimage attacks in both classical and quantum settings. In the parameterized definition, this extra capacity is written as the overhead τ , so that

$$c = d + \tau.$$

The submitted instances use $\tau = 64$, which is word-aligned and covers the maximum supported number of message blocks. Therefore, the 512-, 768-, and 1024-bit versions use capacities of 576, 832, and 1088 bits, respectively.

5.4 Variable-Rate Parameters

The submitted instances use a unified state of $b = 2048$ bits, realized as $n = 32$ words of $w = 64$ bits each. For output length d , the capacity and rate are set as $c = d + \tau$ and $r = b - c$, with $\tau = 64$. Thus, the three output versions use capacities of 576, 832, and 1088 bits, and rates of 1472, 1216, and 960 bits, respectively. The three instances share the same internal permutation, keeping the main implementation structure consistent.

5.5 Multi-Scale Small-Round

Our design of round functions has a multi-scale design. Each large round contains 5 small rounds, and the source-group length used by GM to compute treeSum expands from 2

words to 32 words. Increasing the source-group length increases the source range covered by treeSum. At the same time, the source-group mixing value is fed back through the D_l mapping to a target group, and then participates in the next scale after WR and WP. Thus, multi-scale diffusion is not merely an expansion inside the same group, but a diffusion path formed jointly by treeSum, cross-group feedback, and rearrangement.

From the perspective of attack analysis, small and large strides play different roles. When $l = 2$, one small round contains 16 local source groups. An attacker who wants to control propagation through the entire round must simultaneously control many small treeSum constraints and their target-group feedback. When $l = 32$, the whole state is treated as one source group, and the single treeSum generated by GM acts on all 32 state words, accelerating full-state diffusion. The intermediate strides 4, 8, 16 provide transitions between local-control difficulty and global diffusion speed.

5.6 Tree-Structured Group Mixing and Invertibility

GM is designed to concentrate within-group mixing, nonlinearity, and cross-group diffusion in a local and invertible component. treeSum makes several source words jointly determine the feedback value; modular-addition carries provide bit-level nonlinearity, while XOR branches keep the structure simple. Cross-group feedback moves this local mixing value into a different target group. Fixed-coefficient writeback makes the same feedback value induce different bit-influence patterns in different target words.

GM is an invertible mapping under the parameters in this document. The key facts are that D_l is a permutation on group indices, and each target group is updated by the treeSum of exactly one source group. For each cycle of the permutation D_l , the unknowns can be taken to be the treeSum values of the source groups on the cycle. If one feedback value u is known, then the corresponding target-group input words can be recovered word by word from the output: subtract $\mu_0 u$ from the first word and XOR $\mu_j u$ from the remaining words. Recomputing treeSum on the recovered target group gives the next feedback value on the cycle.

The inverse can recover these feedback values bit by bit from low to high. Suppose that all feedback bits below position k have already been determined. Then all lower-bit effects from modular-addition carries, subtraction borrows, XORs, and fixed-coefficient multiplications are known. At bit k , every recovered word in a target group has the same linear dependence on the current feedback bit u_k . Since l is even, the linear terms of the l word bits cancel in treeSum, so the k -th bit of the next feedback value is determined only by the known output and lower-bit carries. Walking around each permutation cycle therefore recovers all feedback values uniquely, and then all input words can be recovered. The other layers AC, WR, and WP are also invertible, so the complete function f is a permutation.

5.7 Constants in Permutation Function

Round constants enter the state words through AC. In each small round, every group of length l XORs

$$\text{rc} = \alpha[i] \oplus \beta[i_r] \oplus \gamma[I_r]$$

into the first word of that group. Here $\alpha[i]$ distinguishes group indices, $\beta[i_r]$ distinguishes small-round indices, and $\gamma[I_r]$ distinguishes large-round indices. The three classes of constants jointly break symmetries between groups, small rounds, and large rounds. The constants are generated from the mathematical constant π , providing a reproducible source with no hidden selection.

In the GM operation, we need to consider the multiplication constants μ_j and the permutation parameters D_l . For the multiplication constants, we select $\mu_j = 1 + 2^{j+1}$. Each constant has only two non-zero bits, which makes it easy to replace multiplication with shifts and additions. The bit at the lowest position ensures invertibility, and the other non-zero bit differs for each word to avoid a simple non-random property. As for the permutation parameters, we use permutations that consists of a single cycle; therefore, the inverse of the GM operation needs to involve all words. In addition, we ensure that the treeSum of the first half is mapped to the second half, and vice versa. In this way, the state words can be diffused by the uncontrolled part more quickly, thereby increasing the difficulty of the attack.

The rotation constants in WR operation are

$$\rho_i = \left(((7i) \bmod n) + \frac{w-n}{2} \right) \bmod w.$$

Since n is a power of two, 7 and n are coprime, $(7i) \bmod n$ traverses $0, \dots, n-1$ for $i = 0, \dots, n-1$. Thus the n state words use distinct rotation constants. After adding the offset $\frac{w-n}{2}$, the rotation amounts lie in $\frac{w-n}{2}, \dots, \frac{w+n}{2} - 1$, avoiding very small shifts that keep neighboring bit influence too close, as well as rotations close to w that degenerate into very small reverse movements.

In the WP operation, we apply a permutation to shuffle all groups for better diffusion. Under this permutation, all words return to their original positions after $\log_2 n$ small rounds, i.e., one large round, which simplifies implementation and improves performance.

5.8 Innovation of GM Operation

The internal permutation of Iphe uses a multi-scale mixing and diffusion structure. Within one large round, the effective range of local mixing is gradually expanded, while nonlinearity, multi-word mixing, intra-word diffusion, and inter-word rearrangement are combined in a unified word-level framework. Unlike traditional block-cipher-style designs with separate S-boxes and linear layers, Iphe concentrates confusion and diffusion mainly in the word-level permutation centered on GM, keeping the rule compact and easy to implement uniformly.

The forward computation of GM is direct: compute `treeSum` values by group and write them back to target groups. Its inverse, however, is not obtained by simply applying the forward formula backwards word by word. It has to recover feedback values along cycles of D_l , processing bits from low to high so that carries, borrows, and fixed-coefficient multiplications are known when the next bit is determined. The inverse remains linear in the number of state words and the word size, but its description and implementation depend on conditional low-to-high propagation. This combination of a simple forward rule and a nontrivial but provably invertible inverse is a central feature of the internal permutation.

5.9 Simplicity of ARX structure

Iphe is designed using an ARX structure, which means it relies on a small set of word-level operations: modular addition, XOR, ordinary left shift, fixed rotations, and deterministic word permutation. It does not rely on large S-boxes or lookup tables. The four components AC, GM, WR, and WP have clearly separated roles, which makes the specification compact and helps keep the reference implementation close to the mathematical description.

5.10 Flexibility of Parameters

The internal permutation structure used by Iphe is based on a multi-scale small-round design and is structurally extensible. If the construction is extended to an internal state containing $n = 2^k$ words, where $k \in \mathbb{Z}^+$, each large round can contain $n_r = k$ small rounds, with the number of large rounds N_r , round constants, rotation constants, and target-group mappings reselected accordingly. The submitted instances and the security analysis in this document are fixed to $n = 32$, $N_r = 5$, and the concrete constants specified here; the extensibility statement is a structural design feature and is not a security claim for other parameter sets.

Since the core object is a fixed-width permutation, the internal permutation can also be considered as a candidate component for other permutation-based constructions, such as random generation, keyed authentication, or block-cipher-related modes. The concrete instances in this document are still limited to the three hash functions with the output lengths specified above.

5.11 Implementation Features

The main operations of Iphe are 64-bit modular addition, XOR, ordinary left shift, cyclic rotation, and deterministic word rearrangement, which fit common 64-bit software platforms. Except for the reduction structure inside `treeSum`, different source groups in the same small round can compute their `treeSum` values independently; because D_l is a permutation on target groups, different source groups do not write to the same target group. The AC and WR steps are also group-wise or word-wise independent. These properties leave room for multi-word parallelism and limited SIMD vectorization,

although the actual gain depends on the target platform’s support for shifts, cross-lane rearrangement, and register pressure.

In terms of storage, the permutation can be implemented around the 32 64-bit state words, with one 32-word temporary array for GM writeback and WP rearrangement. The absorbing phase only needs to save the capacity part for Sponge-F feed-forward. The implementation does not require large S-boxes or lookup tables, so the static data size is small and a resource-oriented implementation for storage-constrained settings is natural.

6 Security Analysis

6.1 Security Analysis of the Overall Structure

The outer layer adopts the Sponge-F iterative structure proposed in [6]. In the random-permutation model, namely under the assumption that the internal permutation function f is a random permutation, if $c \geq d$, the classical and quantum security bounds for Sponge-F given in [6] are as follows:

- Classical security bounds: the collision security is $d/2$ bits, the preimage security is d bits, and the second-preimage security is $\min\{d, c - \log_2 L\}$ bits, where L is the maximum number of supported message blocks.
- Quantum security bounds: the collision security is $d/4$ bits, the preimage security is $d/2$ bits, and the second-preimage security is $\min\{d/2, (c - \log_2 L)/2\}$ bits, where L is again the maximum number of supported message blocks.

For the detailed proof of the structural security bounds, see [6].

Iphe supports messages of length at most $2^{64} - 1$ bits. Since the minimum rate is 960 bits, the maximum number of message blocks L satisfies $\log_2 L < 55$. The capacity length used in the submitted instances is $c = d + \tau$, with $\tau = 64$. Therefore, for second-preimage attacks, the security bound always satisfies $c - \log_2 L = d + \tau - \log_2 L > d$. Combining the above analysis, the provable classical and quantum security results of the overall structure under the Iphe parameters are shown in Table 4.

Table 4: Provable security bounds for the overall structure of Iphe. The internal permutation f is assumed to be a random permutation.

Security type	Collision	Preimage	Second preimage
Classical security	$\frac{d}{2}$ bits	d bits	d bits
Quantum security	$\frac{d}{4}$ bits [†]	$\frac{d}{2}$ bits	$\frac{d}{2}$ bits

Remark. The entry marked by [†] is the conservative provable lower bound obtained from the Sponge-F analysis of [6]. For a generic d -bit hash output, the quantum collision-search cost is $2^{d/3}$ by the BHT algorithm [8]. Therefore, the claimed generic quantum collision strength of a d -bit digest is $d/3$ bits.

Length-extension resistance. A Merkle–Damgård-style length-extension attack would require using the public digest as the chaining state after processing $m \parallel \text{Padding}(m)$, and then continuing the iteration on attacker-chosen suffix blocks. Iphe does not expose such a chaining state. After the padded message is absorbed, continuing the computation requires the full 2048-bit internal state, including the rate part and the full capacity part used for feed-forward. The digest reveals only d bits from the capacity part. Since $c = d + \tau$ and the submitted instances use $\tau = 64$, one 64-bit capacity word and the entire rate part remain hidden. Therefore, the digest cannot be used as a public chaining value for the standard length-extension attack.

6.2 Security Analysis of the Permutation Function

This subsection analyzes non-random properties of the permutation f used by the algorithm and gives distinguishers for the corresponding round-reduced variants.

6.2.1 Differential, Linear, and Differential-Linear Statistical Analysis

This section reports statistical results on differential cryptanalysis [9], linear cryptanalysis [10], and differential-linear cryptanalysis [11] for round-reduced variants f_R of the internal permutation. Since the 2048-bit state space cannot be exhausted, probabilities and correlations are estimated from random samples, and top candidates are checked with independent samples. The results therefore describe observed reduced-round non-randomness, rather than exhaustive proofs over all input differences or all masks.

For an input XOR difference Δ , define

$$D_R(x, \Delta) = f_R(x) \oplus f_R(x \oplus \Delta).$$

Below, $e_{i,k}$ denotes the unit difference or unit mask whose only set bit is bit k of state word S_i ; the within-word bit numbering follows the 64-bit word interpretation rule defined above. Classical differential cryptanalysis usually considers the probability $\Pr[D_R(x, \Delta) = \nabla]$ of a specified output difference ∇ . The differential-diffusion tests here do not enumerate full output differences; instead, they measure the marginal probability $\Pr[\langle e_{i,k}, D_R(x, \Delta) \rangle = 1]$ for each output bit, which can be viewed as a bit-level truncated-differential statistic. The tests first consider input differences of Hamming weight 1 or 2. The results show that for 3 small rounds, both 1-bit and 2-bit input differences can leave some output bits that never differ. For 4 small rounds, the output bit positions that can differ essentially cover all 2048 bits, but significant probability biases remain. For 5 small rounds, no stable large bias has been observed within the completed screening and verification range. Typical observations are:

- For 1-bit input differences after 4 small rounds, the strongest single-output-bit probability is about 0.688 or 0.312, with bias about 0.188.
- For the 2-bit input difference $\Delta = e_{16,63} \oplus e_{17,63}$, the strongest single-output-bit probability after 4 small rounds is about 0.00795 or 0.99205, so the corresponding

output bit almost never differs or almost always differs. This difference lies in the same $g = 2$ group in the first small round and satisfies $2^{63} + 2^{63} = 0 \bmod 2^{64}$, so the treeSum difference is cancelled and diffusion is slower.

- Several structured 3-bit and 4-bit input differences still show strong 4-small-round biases, but these candidates do not show persistent strong bias after 5 small rounds.

Differential-linear analysis considers an input difference Δ and an output linear mask Γ . Its correlation is defined as

$$\text{Cor}(\Delta \rightarrow \Gamma) = \frac{1}{2^b} \sum_{x \in \mathbb{F}_2^b} (-1)^{\langle \Gamma, f_R(x) \oplus f_R(x \oplus \Delta) \rangle}.$$

If $p_1 = \Pr[\langle \Gamma, D_R(x, \Delta) \rangle = 1]$, then $\text{Cor} = 1 - 2p_1$, and the probability bias is $|p_1 - 1/2| = |\text{Cor}|/2$. The definition allows Γ to be any output linear mask. In the experiments, however, the main tests use input differences of Hamming weight 1 or 2, and output masks formed as the XOR of two output bits. For 4 small rounds, differential-linear relations with probability 0 or 1 exist. For example,

$$\Delta = e_{0,62}, \quad \Gamma = e_{6,38} \oplus e_{7,20},$$

has probability 1 in the observed 4-small-round model. For 5 small rounds, the strongest candidate found in the search,

$$\Delta = e_{0,32} \oplus e_{8,7}, \quad \Gamma = e_{1,51} \oplus e_{25,41},$$

was checked with 2^{26} samples and gave probability 0.49995780 and bias 0.00004220, whose magnitude is about 0.69 sampling standard deviations. Thus, no 5-small-round differential-linear bias that grows stably with the sample size has been observed.

Linear analysis considers

$$C_R(A, B) = \frac{1}{2^b} \sum_{x \in \mathbb{F}_2^b} (-1)^{\langle A, x \rangle \oplus \langle B, f_R(x) \rangle}.$$

If $p_1 = \Pr[\langle A, x \rangle \oplus \langle B, f_R(x) \rangle = 1]$, then $C_R = 1 - 2p_1$, and the probability bias is $|C_R|/2$. The formula allows arbitrary input and output linear masks A and B . The experiments test the structured masks described below and all single-input-bit/single-output-bit mask pairs. For 1 small round, the low-bit group parity relation of GM gives a deterministic affine linear relation; after WR and WP, this relation corresponds to a deterministic output mask. Because of the constant term from AC, the empirical probability can be 0 or 1. For 2 to 5 small rounds, the top candidates from the initial screen all returned to the random-noise range after 2^{22} -sample verification, and no stable multi-small-round linear bias has been observed.

Overall, the reduced-round conclusion supported by these statistics is that differential and differential-linear properties still give strong distinguishing features for 4 small rounds, while no stable bias has been observed for 5 small rounds in the tested range. Linear properties give a direct deterministic relation only for 1 small round, and no stable linear distinguisher extending to multiple small rounds has been observed.

6.2.2 Differential-Neural Analysis

Differential-neural analysis [12], proposed at CRYPTO 2019, builds cryptographic distinguishers using deep neural networks. As a newer analysis method, it has shown advantages over classical analysis methods on some ARX block ciphers. For the R -round permutation function $f_R : \mathbb{F}_2^b \rightarrow \mathbb{F}_2^b$, after choosing an input difference $\Delta \in \mathbb{F}_2^b$, the method trains a deep neural network as a binary classifier. Given a pair of output states $(f_R(x), f_R(x'))$, it distinguishes two classes of samples:

$$Y(f_R(x), f_R(x')) = \begin{cases} 1, & x \oplus x' = \Delta, \\ 0, & (x, x') \text{ are chosen independently at random.} \end{cases}$$

For the target permutation, the attacker constructs training and test sets, each containing roughly half positive samples ($Y = 1$) and half negative samples ($Y = 0$). The neural network is trained on the training set for several epochs and then evaluated on the test set. If the final test accuracy is significantly greater than 0.5, meaning that the neural network performs better than random guessing, an effective distinguisher has been obtained. This is called a differential-neural distinguisher and can be used to distinguish the permutation f_R from a pseudorandom permutation.

This section constructs differential-neural distinguishers for round-reduced versions of the internal permutation f_R . The neural-network architecture uses the convolutional network with residual blocks from [12]. The training and test set sizes are 2×10^6 and 10^5 , respectively. The input difference Δ should be chosen to make the output-distribution bias as large as possible. The tests traverse all input differences with the following form

$$\Delta = e_{i,63}, i \in \{0, 1, \dots, 31\} \quad \text{or} \quad \Delta = e_{2j,63} \oplus e_{2j+1,63}, j \in \{0, 1, \dots, 15\}.$$

These input differences correspond to the most significant bit of each single word and the most significant bits of two consecutive words in each single group in the first small round. For 4 small rounds, many input differences (e.g. $\Delta = e_{0,63}$) led to differential-neural distinguishers with test accuracy 1.0. For 5 small rounds, no valid distinguishers with accuracy significantly higher than 0.5 were obtained. Existing studies show that differential-neural distinguishers are closely related to differential-linear characteristics of the algorithm [13, 14], and the results in this subsection are consistent with the differential-linear analysis in Section 6.2.1. That is, for 4 small rounds differential-linear relations with probability 0 or 1 exist, and for 5 small rounds no differential-linear characteristics with significant bias are observed.

6.2.3 Distinguishing Attacks Based on Algebraic Degree

Write the reduced permutation $f_R : \mathbb{F}_2^{2048} \rightarrow \mathbb{F}_2^{2048}$ as

$$f_R = (F_0, \dots, F_{2047}),$$

where each F_i is one output bit viewed as a Boolean function of all input bits; these functions are the coordinate functions of the permutation. We consider the highest

degree appearing in their ANFs and write

$$\deg(f_R) = \max_i \deg(F_i).$$

The AC step is an affine translation, and WR and WP only permute bits. Hence the degree growth is caused by the carry chains in the modular additions inside GM.

For one small round, a concrete analytic lower bound can be given. Consider the first small round with $g = 2$, and the $k = 0$ feedback branch from the source group (S_0, S_1) to the target word S_{16} . Let the two source words after AC be

$$A = (a_0, \dots, a_{63}), \quad B = (b_0, \dots, b_{63}),$$

and let the target word be

$$X = (x_0, \dots, x_{63}).$$

The target word after GM is

$$Y = X + 3(A + B) \pmod{2^{64}} = X + A + B + (A \ll 1) + (B \ll 1) \pmod{2^{64}}.$$

For the most significant bit Y_{63} , the ANF contains the monomial

$$M = \left(\prod_{i=0}^{61} a_i \right) \left(\prod_{i=1}^{60} b_i \right) \left(\prod_{i=0}^{60} x_i \right)$$

with coefficient 1. Its degree is

$$62 + 60 + 61 = 183.$$

Therefore at least one output coordinate of the first small round satisfies

$$\deg(f_1) \geq 183.$$

The coefficient can be checked by the finite difference corresponding to M . In the weighted-sum representation of Y_{63} , x_i has weight 2^i , while a_i and b_i have weight $3 \cdot 2^i$. The selected variables $x_0, \dots, x_{60}$ give all offsets $0, \dots, 2^{61} - 1$. After the paired a_i, b_i variables are combined in the GF(2) subset count, the nonzero coefficient reduces to the parity of the number of integers q satisfying

$$2^{61} < 3 + 12q < 2^{62}.$$

This number is

$$\left\lfloor \frac{2^{62} - 4}{12} \right\rfloor - \left\lfloor \frac{2^{61} - 3}{12} \right\rfloor = 192153584101141163,$$

which is odd.

Conversely, bit-based degree/support propagation using the full-adder formula gives an upper bound of 188 for one small round. Thus the provable range for the first small round is

$$183 \leq \deg(f_1) \leq 188.$$

The value 188 is an upper bound, not a proof of the exact degree.

For more small rounds, we use restricted-subspace ANF tests and bit-based degree/support propagation. Exhaustive enumeration over structured 26-dimensional affine subspaces proves the presence of degree-26 terms for 1, 2, 3, 4, 5 small rounds and also for the full 25-small-round permutation; this lower bound is limited by the tested dimension. The upper propagation reaches the trivial 2047 bound for permutation coordinate functions by the third small round. In random 24-dimensional subspaces, about half of the output coordinates contain the top-degree term after four to five small rounds, close to the behavior of random Boolean functions. For the inverse permutation f_R^{-1} , the same restricted-subspace ANF test proves degree at least 20 on structured 20-dimensional subspaces for 1, 2, 3, 4, 5 small rounds and for the full 25-small-round inverse; random 20-dimensional subspaces already activate many top-degree output coordinates for inverse 2 to 5 small rounds. Thus, no exploitable low-degree distinguisher has been observed in either the forward or inverse direction.

6.3 Preimage and Second-Preimage Security Analysis

This subsection gives a one-large-round reduced preimage analysis, and the same method directly applies to second-preimage attacks by fixing the target digest of the given message. For the permutation call associated with the last message block, let C and R denote the capacity and rate parts of the input state, and let H be the target digest. Since the digest is taken from the capacity part after the capacity feed-forward, fixing C allows the digest condition to determine the corresponding capacity words immediately before the feed-forward XOR, and these words then give constraints on the intermediate state in the backward analysis.

In the following analysis, an externalized treeSum is a GM treeSum value that is fixed as an independent guess during backward propagation. A candidate rate value is accepted only if its actual treeSum values equal these guesses. The seven treeSums in the third, fourth, and fifth small rounds are always externalized:

$$\text{TS}_8(0), \text{TS}_8(8), \text{TS}_8(16), \text{TS}_8(24), \text{TS}_{16}(0), \text{TS}_{16}(16), \text{TS}_{32}(0).$$

Iphe-768 additionally externalizes

$$\text{TS}_4(8), \text{TS}_4(12), \text{TS}_4(16), \text{TS}_4(20),$$

and Iphe-1024 additionally externalizes

$$\text{TS}_4(4), \text{TS}_4(8), \text{TS}_4(12), \text{TS}_4(16), \text{TS}_4(20).$$

The resulting treeSum guess is denoted by G .

For fixed C , H , and G , backward propagation through the externalized rounds determines the constrained words at the input of the third small round. We denote these words by $A_d(C, G, H)$. Let $F_{C,G}^{(d)}$ be the forward connector from R to the same point,

using the selected second-small-round externalizations when they are present. The SAT connector solves

$$(F_{C,G}^{(d)}(R))_{I_d} = A_d(C, G, H).$$

The constrained index sets are

$$\begin{aligned} I_{512} &= \{6, 7, 14, 15, 22, 23, 29, 30\}, \\ I_{768} &= \{5, 6, 7, 13, 14, 15, 21, 22, 23, 28, 29, 30\}, \\ I_{1024} &= \{4, 5, 6, 7, 12, 13, 14, 15, 20, 21, 22, 23, 27, 28, 29, 30\}. \end{aligned}$$

A returned R is only a connector solution. It becomes a genuine reduced preimage candidate only if the real treeSums are equal to the guess G .

Table 5 reports SAT timing statistics for these connector equations. For each instance, 100 random samples and 100 constructed connectable samples are tested. The random samples choose C , H , and G independently and are usually rejected by the connector. The constructed samples derive C , H , and G from a real state and test the solving cost once the connector is satisfiable. The final equality between the real treeSums and G is counted in the complexity estimate below.

Table 5: SAT timing statistics in the one-large-round reduced preimage analysis

Instance	Solving model	Sample type	Result	Samples	Mean	Median	90% quantile
Iphe-512	fully externalized last-three-round treeSums	random	unsat	100	23.678s	22.446s	38.478s
Iphe-512	fully externalized last-three-round treeSums	constructed connectable	sat	100	23.355s	23.046s	38.289s
Iphe-768	partially externalized round-2 treeSums	random	unsat	100	0.034s	0.032s	0.044s
Iphe-768	partially externalized round-2 treeSums	constructed connectable	sat	100	0.198s	0.195s	0.228s
Iphe-1024	partially externalized round-2 treeSums	random	unsat	100	0.002s	0.002s	0.003s
Iphe-1024	partially externalized round-2 treeSums	constructed connectable	sat	100	0.228s	0.226s	0.254s

For random C , H , and G , the connector is usually unsatisfiable. The effective connector-matching terms used in the reduced estimates are 2^{128} , 2^{320} , and 2^{512} for Iphe-512, Iphe-768, and Iphe-1024, respectively. These matching costs are amortized by explicit neutral families. Let

$$\kappa_i = \alpha[i] \oplus \beta[0] \oplus \gamma[0].$$

For a first-small-round rate pair (w_{2i}, w_{2i+1}) , keep the pair sum

$$s_i = (w_{2i} \oplus \kappa_i) \boxplus w_{2i+1}$$

unchanged by choosing a new word u_i and setting

$$w'_{2i} = u_i \oplus \kappa_i, \quad w'_{2i+1} = s_i \boxplus u_i.$$

This transformation preserves the connector constraints for $i = 0, \dots, 3$ in Iphe-512, for $i = 0, \dots, 8$ in Iphe-768, and for $i = 0, \dots, 6$ in Iphe-1024. Iphe-768 also has one additional neutral word w_{18} in this externalized model. Therefore the explicit neutral spaces are 2^{256} , 2^{640} , and 2^{448} , respectively.

These neutral choices preserve the connector equation, but they do not in general preserve the externalized treeSum values themselves. Hence the final treeSum consistency condition remains 64 bits per externalized word. The part of the connector-matching cost not covered by the neutral family is still counted. Under the uniformity heuristic, the one-large-round reduced preimage complexities for the three submitted instances are therefore 2^{448} , 2^{704} , and $2^{768+(512-448)} = 2^{832}$, respectively.

Table 6: One-large-round reduced preimage complexity summary

Instance	Externalized treeSums	Connector matching term	Explicit neutral space	Complexity
Iphe-512	7 words	2^{128}	2^{256}	2^{448}
Iphe-768	11 words	2^{320}	2^{640}	2^{704}
Iphe-1024	12 words	2^{512}	2^{448}	$2^{768+64} = 2^{832}$

6.4 Collision Security Analysis

For a collision attack, we need to construct differential characteristics that hide all output state word differences from the capacity part. If we can ensure that all treeSum values contain no difference, then we can control the diffusion of differences more easily and obtain valid differential characteristics. Based on the above idea and by choosing suitable input differences to achieve feasible output differences, we present a 2-small-round differential characteristic, which can be applied to all versions of Iphe. For two states S_i and S'_i , we describe our differential characteristic in Table 7.

Table 7: 2-Small-Round differential characteristic

Stage	Differences
Before the first small round	$S'_8 \boxminus S_8 = 2^{59}$, $S'_9 \boxminus S_9 = 0 \boxminus 2^{59}$, $S'_{12} \boxminus S_{12} = 2^{63}$, and $S'_{13} \boxminus S_{13} = 2^{63}$
After GM of the first small round	$S'_8 \oplus S_8 = 2^{59}$, $S'_9 \oplus S_9 = 2^{59}$, $S'_{12} \oplus S_{12} = 2^{63}$, and $S'_{13} \oplus S_{13} = 2^{63}$
After WR of the first small round	$S'_8 \oplus S_8 = 2^{35}$, $S'_9 \oplus S_9 = 2^{42}$, $S'_{12} \oplus S_{12} = 2^{35}$, and $S'_{13} \oplus S_{13} = 2^{42}$
After WP of the first small round	$S'_4 \oplus S_4 = 2^{35}$, $S'_{20} \oplus S_{20} = 2^{42}$, $S'_6 \oplus S_6 = 2^{35}$, and $S'_{22} \oplus S_{22} = 2^{42}$
After GM of the second small round	Only S_4 , S_6 , S_{20} , and S_{22} contain non-zero differences
After WR of the second small round	Only S_4 , S_6 , S_{20} , and S_{22} contain non-zero differences
After WP of the second small round	Only S_2 , S_3 , S_{10} , and S_{11} contain non-zero differences

For the first small round, we can choose the input state words so as to ensure proper difference propagation. For the second small round, we need 4 conditions to control the 4 XOR differences of TS₂; hence, the probability of this differential characteristic is about 2^{-4} . Since, for all versions of Iphe, the input differences are located in the rate part and the output differences are hidden from the capacity part, this differential characteristic can be used to mount collision attacks.

However, for more small rounds, if we also want to keep all treeSum values free of differences, the degrees of freedom become insufficient to construct suitable differential characteristics. For example, if we introduce differences into input state words S_0 , S_1 , S_2 , S_3 , S_8 , S_9 , S_{10} , and S_{11} , then in 3 small rounds we would need to ensure that 12 treeSum values contain no difference, but we only have 8 non-zero input state word differences.

The SAT solver returns UNSAT, indicating that no such differential characteristic exists. Since the main limitation is the lack of degrees of freedom rather than computational complexity, it is also impossible to mount a quantum attack by this method on more rounds of Iphe.

6.5 Statistical Randomness Tests

This section reports statistical randomness tests for the complete Iphe hash algorithm. The test implementation follows the 15 items in the GM/T 0005-2021 Randomness Test Specification: monobit frequency, block frequency, poker, overlapping serial patterns, total runs, runs distribution, longest run in a block, binary derivative, autocorrelation, matrix rank, cumulative sums, approximate entropy, linear complexity, Maurer’s universal statistical test, and discrete Fourier transform. We use a single-sequence test setting: for each instance and each message-generation mode, many digests are concatenated into a 2^{20} -bit sample stream. A test item is considered to pass when its p -value is at least 0.01. This experiment is a statistical check of the digest stream and is not the 1000-sample pass-rate procedure of the GM/T standard.

The following input sources are tested. Except for **low** and **high**, the inputs are processed by the complete hash algorithm, including the specified padding rule. To avoid mixing fixed padding blocks into the low-density and high-density samples, **low** and **high** are not treated as variable-length messages. Instead, a 2048-bit full-state input is generated directly, the core permutation is called once, and digest bits are taken from the corresponding capacity positions.

- **random**: a 1024-bit random message.
- **low**: a 2048-bit low-density full-state input with about 1% one bits, without padding.
- **high**: a 2048-bit high-density full-state input with about 1% zero bits, without padding.
- **counter**: a 64-bit counter processed with the padding rule.
- **feedback**: an output-feedback message chain. For digest length d , set $m_0 = 0^d$ and recursively set $m_{i+1} = \text{Hash}(m_i, d)$. The tested stream is $\text{Hash}(m_0, d) \parallel \text{Hash}(m_1, d) \parallel \dots$.

Table 8 gives supplementary confirmation results for 2^{20} -bit sample streams. Each row contains the 15 test items; a result of 15/15 means that all items pass at the selected significance level.

Table 8: 2^{20} -bit single-sequence statistical randomness confirmation results

Version	Message mode	Passed items	Minimum p -value
512	random	15/15	1.937×10^{-1}
512	low	15/15	1.160×10^{-1}
512	high	15/15	1.689×10^{-1}
512	counter	15/15	1.262×10^{-1}
512	feedback	15/15	1.632×10^{-1}
768	random	15/15	6.140×10^{-2}
768	low	15/15	4.497×10^{-2}
768	high	15/15	1.643×10^{-1}
768	counter	15/15	1.235×10^{-1}
768	feedback ¹	15/15	2.359×10^{-1}
1024	random	15/15	3.109×10^{-2}
1024	low	15/15	1.922×10^{-2}
1024	high	15/15	3.501×10^{-2}
1024	counter	15/15	2.546×10^{-2}
1024	feedback	15/15	1.953×10^{-1}

In the first test, two individual items had $p < 0.01$: the overlapping-serial test for the initial Iphe-768 **feedback** window and the longest-run-in-a-block test for Iphe-1024 with **low** full-state input. Since each individual test uses significance level 0.01, a small number of low p -values is expected when many samples and test items are inspected simultaneously. Such observations therefore need to be checked for stability on supplementary samples, later feedback windows, or longer sample streams.

The follow-up tests show no stable failure for the Iphe-1024 **low** input; the low p -value items for low-density and high-density full-state inputs do not concentrate on a fixed test item. For Iphe-768 **feedback**, the low p -value in the overlapping-serial test repeats on the same initial window, but later 2^{20} -bit windows from the same feedback stream pass this test. Longer-prefix tests also do not reproduce this failure. Overall, these GM/T-style single-sequence tests do not show systematic statistical-randomness failures for Iphe; the initial Iphe-768 **feedback** window and the Iphe-768 **random** overlapping-serial behavior remain points for larger follow-up tests.

7 Implementation

7.1 Overview

The Iphe submission provides three software implementations with distinct engineering objectives. Under the common directory `submit/Implementations/`, they are organized as follows:

¹The Iphe-768 **feedback** row uses a later 2^{20} -bit window from the same fixed feedback stream; the all-zero initial feedback window is discussed below.

- **Reference_Implementation:** a readable correctness baseline;
- **Optimized_Implementation:** a throughput-oriented version; and
- **Additional_Implementation:** a resource-oriented version.

Each version contains separate directories for Iphe-512, Iphe-768, and Iphe-1024. Known-answer test files are stored under `submit/Test_Vectors/`. All three versions expose the `API_CryptHash` interface specified by the Commercial Cryptography Standards Research Institute and require no third-party cryptographic library.

The internal state consists of 32 64-bit words, for a total of 2048 bits. All three instances use the same 2048-bit permutation. Their principal differences are the digest length, rate, and capacity. Processing a message block consists of XOR absorption into the rate portion, one permutation call, and capacity feed-forward. Finalization then squeezes the digest directly from the capacity portion. For the three fixed digest lengths specified here, the final squeeze requires no additional permutation call.

7.2 Software Implementation

The Reference implementation follows the algorithm description faithfully, and favors human readability, and supports correctness checking, with direct comparison with the specification file.

The Optimized implementation targets x86-64 throughput. It has C-level optimization of critical paths, loop unrolling, link-time optimization, and AVX2 code-generation options. These choices reduce loop, indexing, and call overhead while exposing more instruction-level parallelism. The Resource implementation uses unified loops, compact structure, and size optimization to reduce code and static-memory footprint. The Performance and Resource versions therefore provide explicit alternatives in the speed-code-size trade-off.

No version uses a large S-box or a large lookup table. The principal operations are 64-bit modular addition, XOR, ordinary left shift, fixed rotation, and word permutation. These operations map naturally to 64-bit processors. Long-message performance is governed mainly by the recurring permutation cost, whereas short-message measurements include a larger contribution from interface, initialization, padding, finalization, and timing overhead.

7.3 Theoretical Cost

In this section we count the number of operations in the Iphe hash function. The main cost is from the permutation function invoked once per absorbed message block. The instance rate determines how many message bytes are processed by each permutation call. For the same implementation strategy and input length, an instance with a larger rate consequently requires fewer permutation calls and is expected to achieve higher long-message throughput. This distinction directly affects Iphe-512, Iphe-768, and Iphe-1024 because their rates are different. Rate-part absorption, capacity feed-forward, padding,

and final digest extraction are linear auxiliary steps. Their contribution to long-message cost is smaller than that of the permutation.

Let $n = 32$ denote the number of 64-bit state words. One full permutation contains five large rounds. Within each large round, the group-mixing stages use

$$l \in \{2, 4, 8, 16, 32\}.$$

The number of groups accumulated across these stages is

$$\sum_{l \in \{2, 4, 8, 16, 32\}} \frac{n}{l} = 16 + 8 + 4 + 2 + 1 = 31.$$

Accounting for all operations in AC, GM, WR, and WP gives the word-level counts in Table 9.

Table 9: Word-level cost of one Iphe permutation.

Operation	Count
64-bit modular additions	1420
Ordinary left shifts	645
Word XORs	980
Fixed word rotations	800
Word moves in WP	800

The permutation is consequently the dominant per-block cost. Absorption adds only $r/64$ word XORs per full block. Saving and restoring the capacity for feed-forward and copying the final digest are comparatively small linear operations. The rates of Iphe-512, Iphe-768, and Iphe-1024 are 1472, 1216, and 960 bits, corresponding to 184, 152, and 120 bytes. A larger rate processes more input per permutation call and is therefore expected to provide higher throughput for sufficiently long messages.

7.4 Benchmark Environment

Tables 10 and 11 summarize the measurement platform and benchmark setup.

The three versions were compiled with the following options:

Reference `-std=c99 -Wpedantic -Wall -Wextra -O2.`

Performance `-O3 -march=x86-64 -mavx2 -mtune=native -flto
-ffat-lto-objects -fomit-frame-pointer -std=c99
-Wpedantic -Wall -Wextra.`

Resource `-O0 -march=x86-64 -mavx2 -flto -ffat-lto-objects
-fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra.`

Table 10: Benchmark hardware and operating system.

Item	Configuration
Processor	Intel(R) Core(TM) Ultra 7 251HX
Frequency	2900 MHz
Cores / logical processors	18 / 18
Memory	15.43 GB
Operating system	Windows 11 Home, Chinese Edition
Version	25H2, OS build 26200.8655
Architecture	x86_64 / 64-bit

Table 11: Software environment and benchmark setup.

Item	Configuration
Programming language	ISO C, C99 dialect
Environment	MSYS2 UCRT64
MSYS2 runtime	<code>msys2-runtime 3.6.9-1</code>
Compiler	<code>gcc.exe</code> , Rev5, MSYS2 project, version 16.1.0
Build tool	CMake 4.3.2
Instruction set	x86-64; AVX2 enabled for optimized builds
Cryptographic library	None; the Iphe implementation itself does not depend on external cryptographic libraries
Benchmark framework	NGCC
CPU affinity	Logical CPU 0, affinity mask 0x1
Cycle counter	x86 <code>__rdtscp</code>
Repetitions	1000 per test point

Average cycle counts were obtained from the x86 `__rdtscp` timestamp counter. They represent average TSC ticks rather than hardware-PMU core cycles. NGCC reports raw throughput in Mbps; the tables below use MB/s according to

$$\text{MB/s} = \frac{\text{Mbps}}{8}.$$

CPU pinning reduces migration noise, but Windows scheduling, background activity, dynamic frequency changes, and cache state can still produce local variation.

7.5 Software Benchmark Results

Across S1–S8, performance generally improves and then approaches a stable region as the input length increases, although local non-monotonic points can occur. Repeated testing found no evidence of implementation errors. Local fluctuation is primarily attributable to rate-based block processing, padding-boundary effects, amortization of fixed overhead, and Windows timing and scheduling noise. The functional tests remain PASS.

Tables 12–14 group the three instances by implementation version. This arrangement keeps each caption with its data and reduces the number of floating objects.

7.6 Resource Consumption

The static section size is obtained by parsing the native object or static library sections. The stack peak is derived from GCC `-fstack-usage` output and the deepest call chain. The implementation-level memory upper bound is therefore

$$M_{\text{impl}} = M_{\text{static section}} + M_{\text{stack}}.$$

The Working Set and Commit peaks apply to the complete NGCC test process. They are auxiliary process-level observations and are not used as the algorithm’s implementation-level peak memory.

The static-section measurements are listed in Table 15. Table 16 reports the stack static-analysis values and the resulting upper bounds. Splitting these data keeps both tables within a normal text width.

Table 12: Reference implementation benchmark results.

Instance	Input bytes	Average cycles	Throughput (MB/s)
Iphe-512	32	21834.24	4.78
	128	20325.98	20.81
	512	58151.27	27.20
	1024	66257.57	45.56
	4096	261375.95	43.40
	8192	517372.93	47.25
	16384	1037635.31	51.75
	65536	4104126.36	53.33
Iphe-768	32	18641.83	5.54
	128	20806.92	20.59
	512	83206.00	20.45
	1024	98106.95	21.01
	4096	289071.29	36.66
	8192	630631.72	36.28
	16384	1231739.95	44.66
	65536	4928911.95	43.88
Iphe-1024	32	18623.72	5.30
	128	41396.33	10.16
	512	75636.95	17.45
	1024	111213.40	27.89
	4096	372563.84	27.30
	8192	776440.08	28.99
	16384	1528935.18	34.71
	65536	9201642.62	35.06

Table 13: Performance-oriented implementation benchmark results.

Instance	Input bytes	Average cycles	Throughput (MB/s)
Iphe-512	32	3962.07	27.88
	128	2428.17	179.35
	512	5418.43	263.54
	1024	11205.75	324.93
	4096	37461.11	361.72
	8192	70300.08	373.88
	16384	106427.96	443.16
	65536	305694.60	542.13
Iphe-768	32	2370.99	44.66
	128	1939.02	228.86
	512	7714.28	214.86
	1024	12074.24	292.81
	4096	39321.22	314.62
	8192	77269.92	317.98
	16384	113428.23	345.70
	65536	389442.20	492.34
Iphe-1024	32	2747.26	39.48
	128	4530.16	95.30
	512	9084.15	179.78
	1024	14971.29	229.91
	4096	55629.43	242.38
	8192	92682.46	248.05
	16384	128970.85	301.56
	65536	444685.65	415.85

Table 14: Resource-oriented implementation benchmark results.

Instance	Input bytes	Average cycles	Throughput (MB/s)
Iphe-512	32	29282.26	3.54
	128	15983.97	17.37
	512	60604.08	24.80
	1024	117523.58	21.30
	4096	344589.67	24.08
	8192	684141.64	36.40
	16384	1365396.81	37.76
	65536	6933173.62	38.96
Iphe-768	32	28677.92	2.73
	128	27614.03	15.00
	512	97763.56	14.89
	1024	134933.57	16.65
	4096	416841.95	29.09
	8192	834268.06	28.54
	16384	1785009.21	33.15
	65536	11598094.41	31.41
Iphe-1024	32	29344.46	3.63
	128	53347.75	7.65
	512	107852.93	12.50
	1024	154631.71	15.56
	4096	545376.15	23.76
	8192	1051152.16	25.21
	16384	2104623.56	26.05
	65536	16585060.64	25.51

Table 15: Static section sizes in bytes.

Version	Instance	Text	Data	BSS	Total
Reference	Iphe-512	1536	372	0	1908
Reference	Iphe-768	1536	372	0	1908
Reference	Iphe-1024	1600	372	0	1972
Performance	Iphe-512	6944	704	0	7648
Performance	Iphe-768	7040	704	0	7744
Performance	Iphe-1024	8416	808	0	9224
Resource	Iphe-512	912	336	0	1248
Resource	Iphe-768	912	336	0	1248
Resource	Iphe-1024	928	336	0	1264

Table 16: Stack analysis and implementation-level memory upper bounds.

Version	Instance	Stack (B)	M_{impl} (B)
Reference	Iphe-512	1408	3316
Reference	Iphe-768	1392	3300
Reference	Iphe-1024	1392	3364
Performance	Iphe-512	1024	8672
Performance	Iphe-768	1008	8752
Performance	Iphe-1024	1088	10312
Resource	Iphe-512	1120	2368
Resource	Iphe-768	1120	2368
Resource	Iphe-1024	1120	2384

Table 17: Auxiliary process-level NGCC memory observations.

Version	Instance	Working Set peak (B)	Commit peak (B)
Reference	Iphe-512	2555904	454656
Reference	Iphe-768	2555904	458752
Reference	Iphe-1024	3715072	692224
Performance	Iphe-512	4403200	733184
Performance	Iphe-768	2555904	454656
Performance	Iphe-1024	2555904	454656
Resource	Iphe-512	2551808	454656
Resource	Iphe-768	2551808	458752
Resource	Iphe-1024	2555904	458752

References

- [1] Bertoni Guido, Daemen Joan, P Michaël, and V Gilles. Cryptographic sponge functions. *2011-STMicroelectronics NXP Semiconductors, Version 0.1 January 14*, 2011.
- [2] Morris J Dworkin et al. Sha-3 standard: Permutation-based hash and extendable-output functions. 2015.
- [3] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, pages 313–314, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [4] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34, 07 2021.
- [5] Daniel J. Bernstein, Stefan Kölbl, Stefan Lucks, Pedro Maat Costa Massolino, Florian Mendel, Kashif Nawaz, Tobias Schneider, Peter Schwabe, François-Xavier Standaert, Yosuke Todo, and Benoît Viguier. Gimli : A cross-platform permutation. In Wieland Fischer and Naofumi Homma, editors, *Cryptographic Hardware and Embedded Systems – CHES 2017*, pages 299–320, Cham, 2017. Springer International Publishing.
- [6] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. *Cryptology ePrint Archive*, 2025.
- [7] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. *Cryptology ePrint Archive*, 2025.
- [8] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *Latin American Symposium on Theoretical Informatics*, pages 163–169. Springer, 1998.
- [9] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *Journal of CRYPTOLOGY*, 4(1):3–72, 1991.
- [10] Mitsuru Matsui. Linear cryptanalysis method for des cipher. In Tor Helleseth, editor, *Advances in Cryptology — EUROCRYPT ’93*, pages 386–397, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
- [11] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo G. Desmedt, editor, *Advances in Cryptology — CRYPTO ’94*, pages 17–25, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.

- [12] Aron Gohr. Improving attacks on round-reduced speck32/64 using deep learning. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 150–179, Cham, 2019. Springer International Publishing.
- [13] Yi Chen and Hongbo Yu. Bridging machine learning and cryptanalysis via EDLCT. Cryptology ePrint Archive, Paper 2021/705, 2021.
- [14] Adrien Benamira, David Gerault, Thomas Peyrin, and Quan Quan Tan. A deeper look at machine learning-based cryptanalysis. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021*, pages 805–835, Cham, 2021. Springer International Publishing.